

IDE

IDE

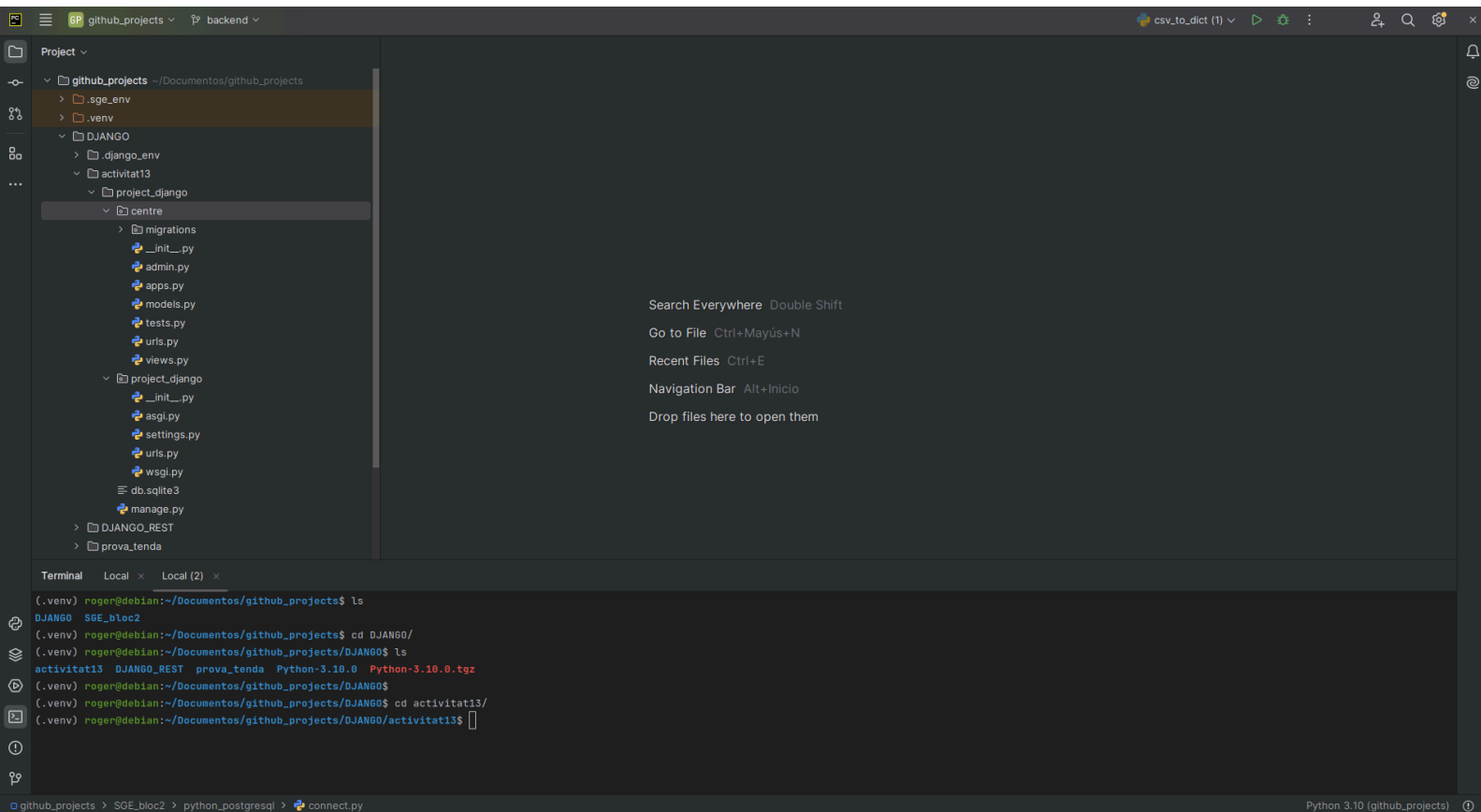
IDE que es recomana per tal d'optar a una instal·lació ràpida i senzilla tant en windows com en linux. Es recomana utilitzar **PyCharm**.

Les següents passes serveixen tant per windows com per linux.

DESCARREGAR I INSTAL·LAR PYCHARM

[Descarregar](#) la versió **community** segons el SO.

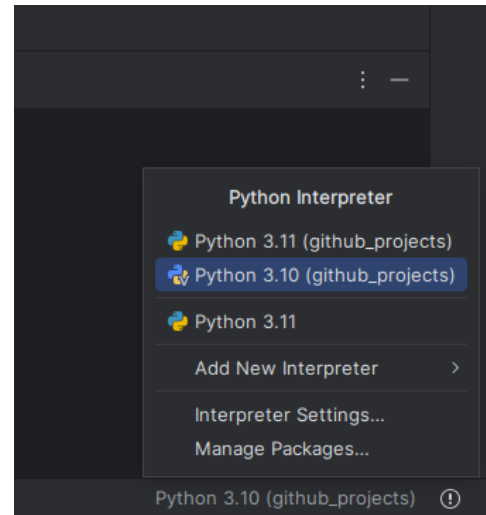
Instal·lar Pycharm



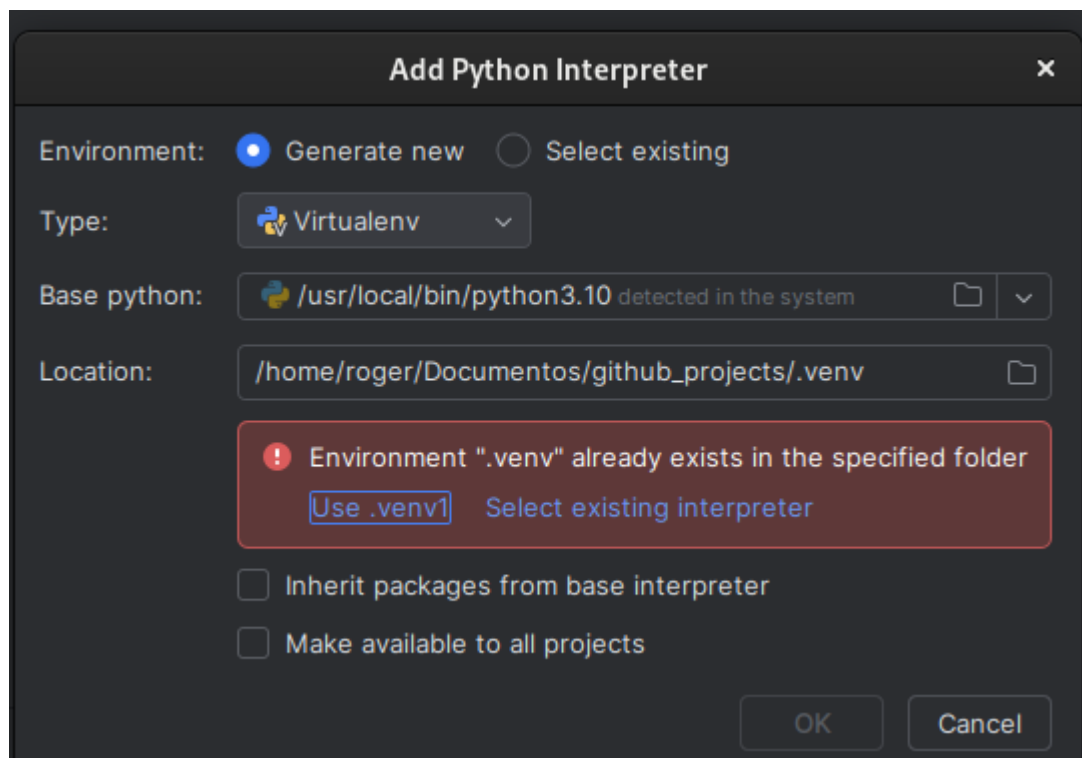
PYTHON INTERPRETER

A la part inferior dreta caldrà buscar l'interpreter de Python (**Python 3.10**) més estable pels projectes a treballar.

Si no es troba, caldrà descarregar la versió 3.10 per a que PyCharm el detecti i s'hi pugui treballar.



Si no detecta la versió 3.10 caldrà anar a l'opció **add new interpreter** i buscar la versió 3.10



Si seleccioneu a **type** virtualenv, Pycharm crearà un entorn virtual amb **virtualenv** amb Python3.10 i totes les llibreries que s'instal·lin seran segons la versió de l'entorn virtual.

Sempre que es treballi amb PyCharm utilitzar l'entorn virtual creat.

Activar entorn virtual

Terminal (Linux)

```
source ./nom_env/bin/activate
```

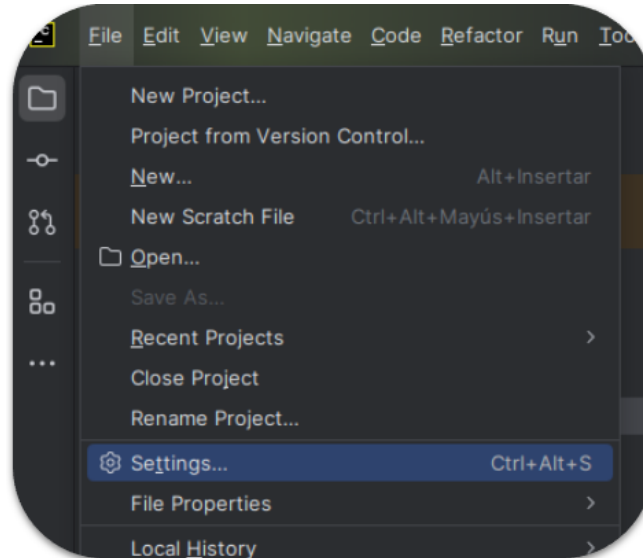
Terminal (windows)

```
.\env_name\Scripts\activate
```

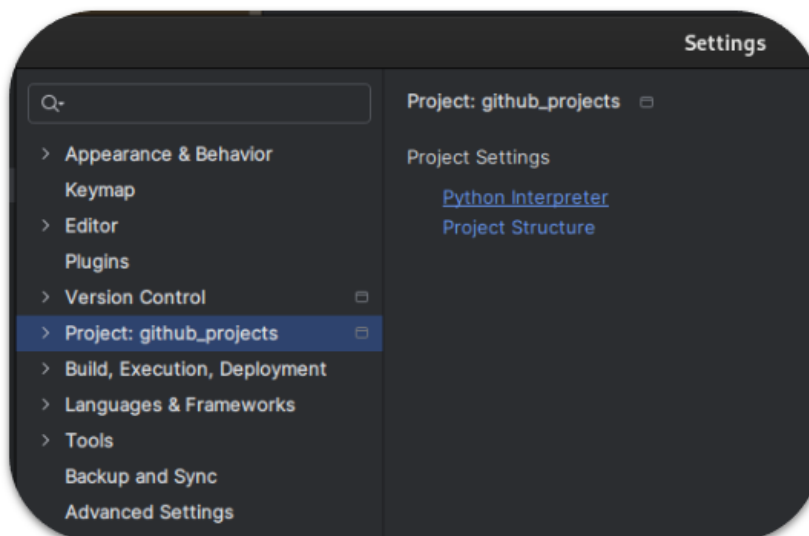
INSTALAR PAQUETS A PYCHARM

Per instal·lar llibreries en l'entorn de treball seguir el següent procés:

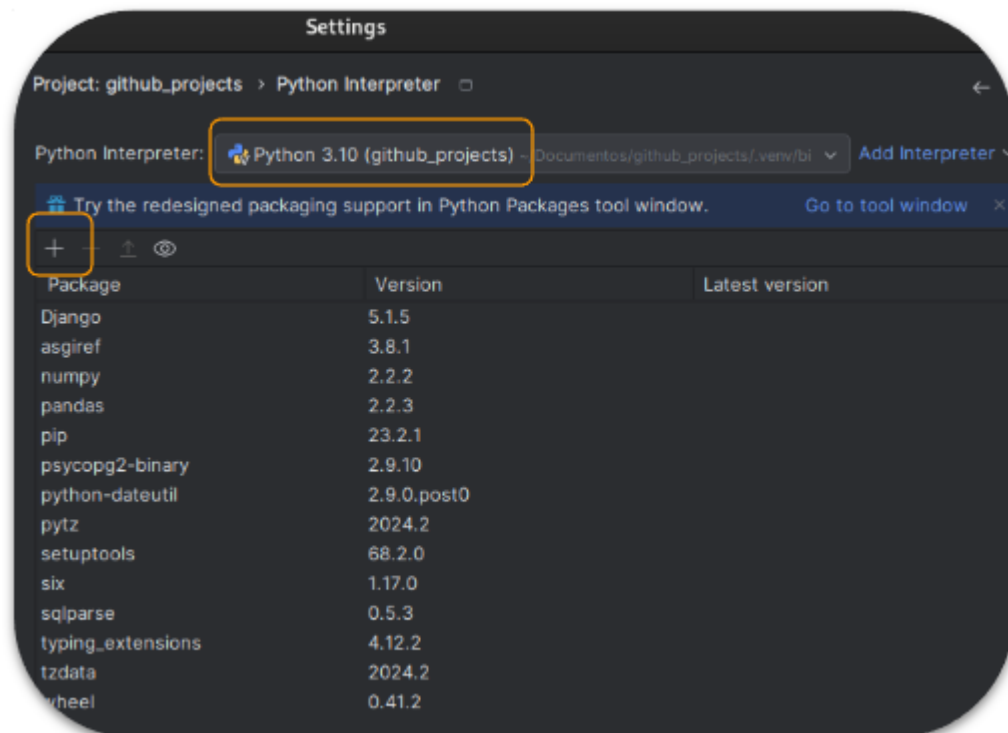
- 1) Anar al menu d'opcions de la part superior i buscar **File > Settings**



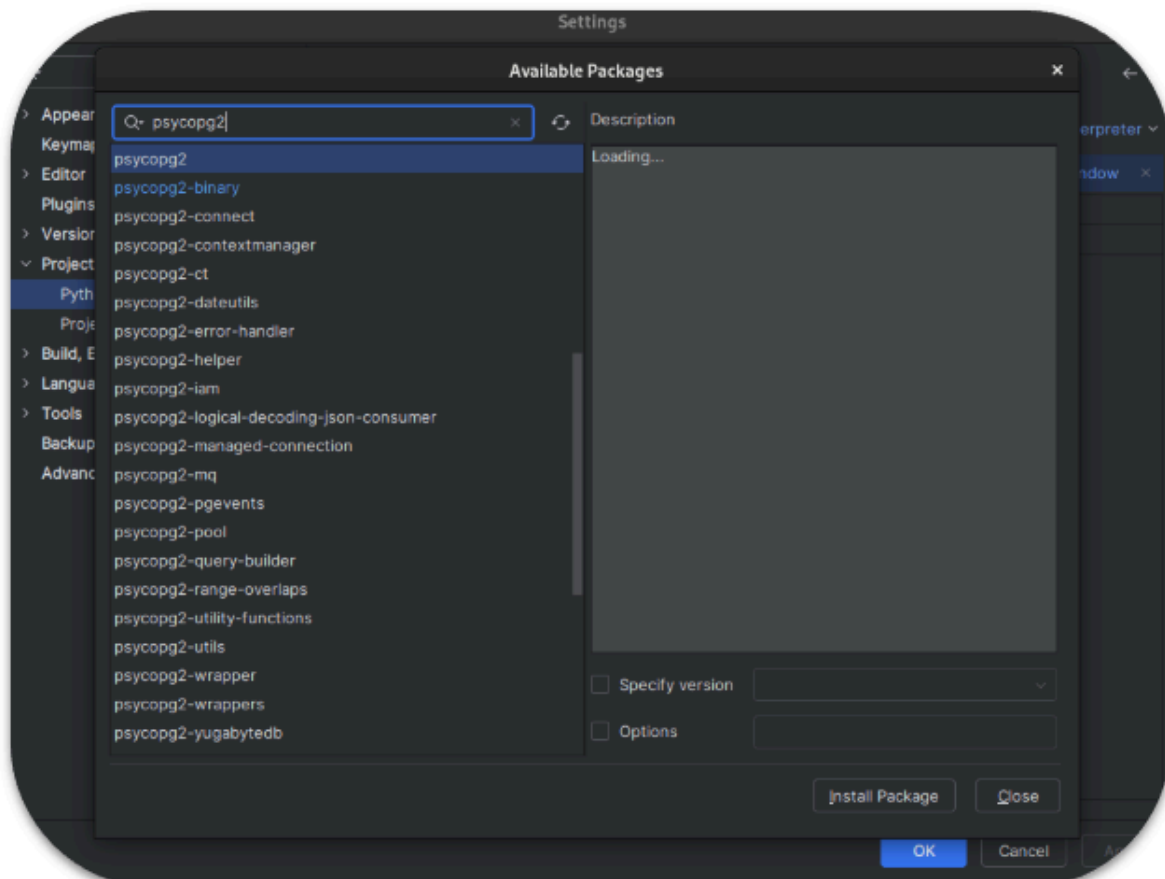
- 2) Buscar **Project:... > Python Interpreter**.



- 3) Us obrirà un panell d'opcions. Abans d'afegir res, mirar sobretot que a **Python interpreter** hi tinguis la versió 3.10. Sempre que afegiu una nova llibreria s'instal·larà segons la versió que hi tingueu en aquella part afegida.



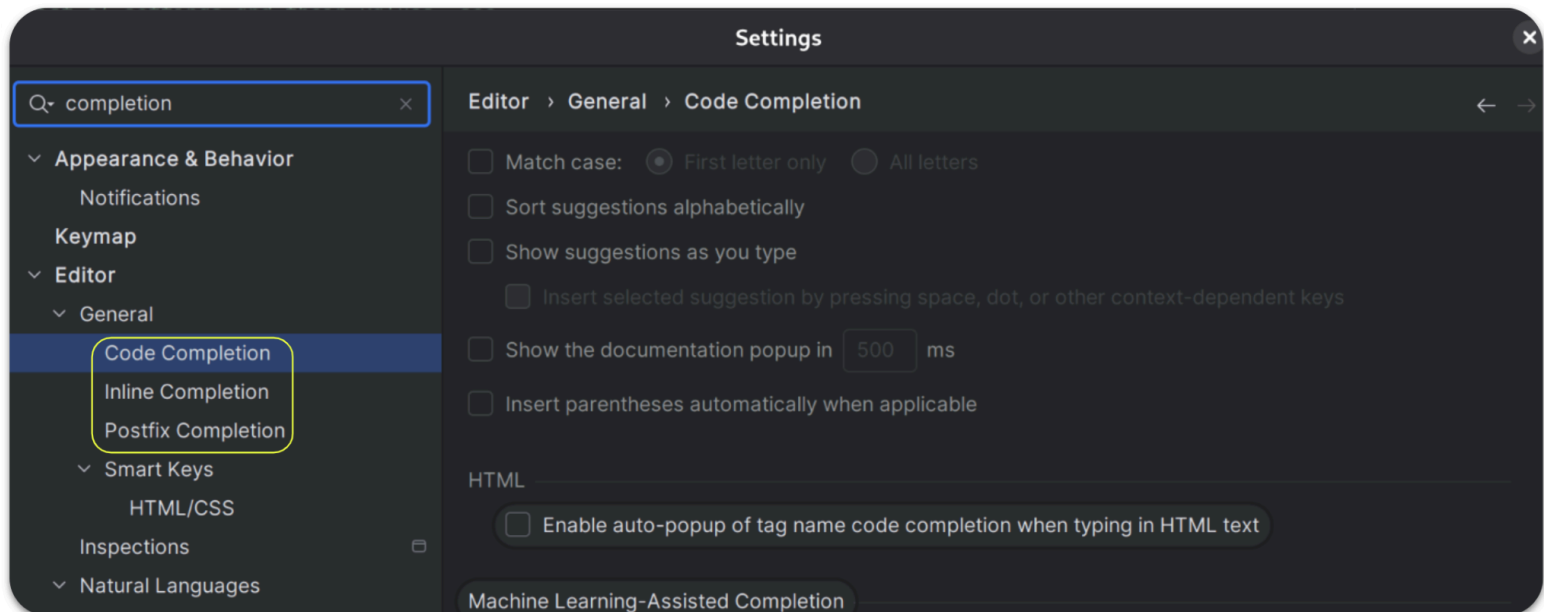
- 4) Apretar a la opció + per tal de buscar llibreria i apretar **install package**.



En cas d'optar per utilitzar un altre IDE que no sigui PyCharm, començar amb [PREPARACIÓ ENTORN](#).

En cas d'haver seguit aquestes passes i després de verificar el bon funcionament, començar amb [DOCKER](#).

Es demana (igual que es demanarà als exàmens o EI) que es desactivi **completion** (autocompletat de PyCharm).



PREPARACIÓ ENTORN

Abans d'utilitzar DJANGO cal preparar l'entorn. Per a la preparació de l'entorn cal saber en quin Sistema Operatiu S'instal·larà el framework. Com a consulta es pot caldrà anar mirant la [documentació oficial](#).

En totes les captures i exemples que es puguin anar indicant amb captures en els documents referenciats al projecte de DJANGO es faran en base a consultes a la BD de POSTGRESQL. Per tal de funcionar caldrà instal·lar la llibreria de **psycopg2** (amb versió python 3.10). En cas d'utilitzar altres BD seguir les recomanacions de la documentació oficial de [instal·lació de DB](#).

Pels exemples / captures mostrats/des en aquests documents, es faran sobre el Sistema Operatiu DEBIAN (última versió publicada al 2025).

Es recomana, abans d'instal·lar res, crear un entorn virtual amb la següent comanda:

Terminal

```
python3.10 -m pip install virtualenv
```

i crear l'entorn amb una versió de python de 3.10.

Terminal

```
python3.10 -m venv <nom_env>
```

Caldrà instal·lar **psycopg2** per treballar amb **postgresql**.

Terminal

```
python3.10 -m pip install psycopg2
```

Posar la comanda per terminal per instal·lar la versió de django adequada.

Terminal

```
sudo python3 -m pip install Django==4.1
```

o

Terminal

```
sudo pip install Django==4.1
```

Extret de la documentació oficial per la [instal·lació](#).

IMPORTANT!!

Sigueu conscients de que copiar i enganxar, sense escriure-ho i sense mirar el feedback de la pantalla, us pot portar a fer una instal·lació errònia.

INSTALAR PYTHON 3.10

INSTAL·LACIÓ PYTHON 3.10 AMB LINUX

Instal·lar dependències

Terminal

```
sudo apt update  
sudo apt install build-essential zlib1g-dev libncurses5-dev libgdbm-dev libnss3-dev libssl-dev  
libreadline-dev libffi-dev libsqlite3-dev wget libbz2-dev
```

Descarregar el còdi font de Python 3.10

Terminal

```
wget https://www.python.org/ftp/python/3.10.0/Python-3.10.0.tgz  
tar -xvf Python-3.10.0.tgz  
cd Python-3.10.0
```

Configurar i compilar Python 3.10

Terminal

```
sudo ./configure --enable-optimizations  
sudo make -j 2  
sudo make altinstall
```

Verificar la instal·lació

Terminal

```
python3.10 --version
```

DOCKER

ENLLAÇOS:

Documentació oficial: [Docker docs](#)
[DockerHub](#)

PAS A PAS

- INSTAL·LACIÓ

Seguir les pases d'instal·lació segons el que s'indica a la [documentació oficial](#) i segons el SO.

Ha de quedar clar que si s'utilitza Windows és obligatori instal·lar **docker desktop** i sempre que es vulgui treballar amb les comandes del docker, cal tindre obert el **docker desktop**.

Amb Linux, només cal instal·lar **docker engine** segons la versió de linux.

- ARXIU DOCKER-COMPOSE.YML

Una vegada hagueu confirmat que la instal·lació del docker és correcte i funcional, crear un arxiu de nom **docker-compose.yml** (dintre d'una carpeta).

Terminal

```
mkdir docker  
code docker-compose.yml
```

Amb l'IDE que us vagi millor obrir l'arxiu creat i posar el següent codi:

`docker-compose.yml`

```
version: '3.1'
services:
  db:
    image: postgres:13
    container_name: db_UF3
    environment:
      - POSTGRES_DB=django
      - POSTGRES_PASSWORD=admin
      - POSTGRES_USER=admin
    ports:
      - "5432:5432"
    volumes:
      - local_pgdata:/var/lib/postgresql/data
  pgadmin:
    image: dpage/pgadmin4
    container_name: pg_UF3
    ports:
      - "80:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: roger@roger.com
      PGADMIN_DEFAULT_PASSWORD: admin
    volumes:
      - pgadmin-data:/var/lib/pgadmin
volumes:
  local_pgdata:
  pgadmin-data:
```

Amb aquest arxiu s'hi estan creant 2 serveis (**db** i **pgadmin**). En la secció d'**environment** s'hi indiquen les dades per entrar als serveis.

Una vegada activats els serveis (mirar **Comandes**) ja no es pot modificar les dades a **environment** a no ser que s'eliminin els **volume**.

COMANDES

- Activar serveis indicats a l'arxiu **docker-compose.yml**. En el mateix directori de l'arxiu posar:
sudo docker-compose up -d
o
sudo docker compose up -d
- Veure contenidors (dockers) actius:
sudo docker ps
- Veure contenidors actius i no actius:
sudo docker ps -a
- Parar contenidors:
sudo docker stop <nom contenidor>
- Eliminar contenidors:
sudo docker rm <nom contenidor>
- Veure volums creats:
sudo docker volume ls
- Eliminar volums:
sudo docker volume rm <nom volume>
- Localitzar i eliminar PORT actiu:
sudo lsof -i:5432 (canviar 5432 pel port que es vulgui buscar)
sudo kill <num PID del port>

DJANGO MVT

DJANGO



1. Introducció a Django ➡

Django és un framework de **Python** per crear projectes de front-end complets que segueix el patró de disseny **MVT (Model View Template)** és semblant al patró MVC simplement que al patró MVT, la V és el nostre controlador (p. ex.: main.py en fastapi) i la T és la renderització del front-end.

Model: Gestiona les dades de la Base de Dades mitjançant l'**ORM** (Object Relational Mapping). Es troba a **models.py**.

View (Vista): És el controlador que gestiona els mètodes i truca un Template segons la demanda de l'usuari. Es troba a **views.py**.

Template (Plantilla): Arxiu (HTML) que conté el disseny de la pàgina web. Treballa amb **tags** de django. S'ubica a la carpeta **templates**. Aquesta carpeta cal crear-la.

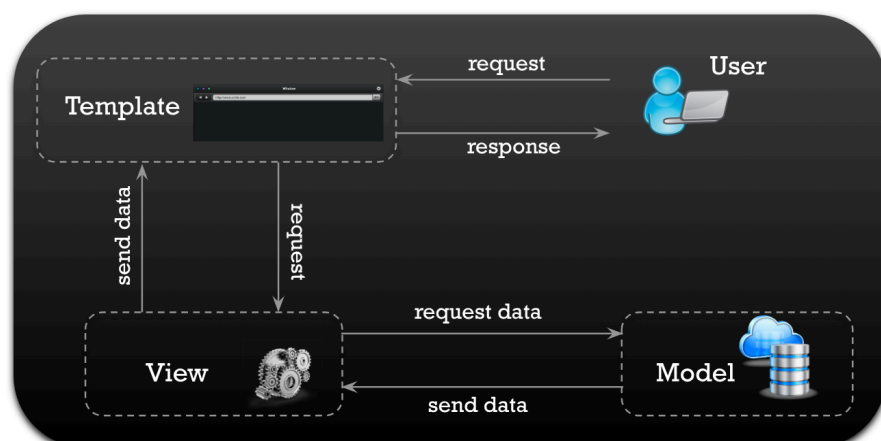
Django treballa per **aplicacions** dins d'un projecte. Per exemple, una **botiga** (projecte) en línia pot tenir aplicacions independents per a: **gestió de productes, compres i pagaments**.

2. Funcionament i Flux de Treball

Quan es fa una consulta en un projecte Django:

- 1. Enrutament:** El projecte rep la petició i cerca la coincidència a **urls.py**.
- 2. Orquestració:** La Vista (**views.py**) rep la petició. Si el disseny és complex, aquí és on cridaria el **service** corresponent.
- 3. Dades:** S'obtenen o modifiquen les dades a través dels **Models** (usant l'ORM).
- 4. Presentació:** La vista recull el resultat i **renderitza el Template** (HTML) passant-li el context (les dades).
- 5. Resposta:** S'envia el fitxer HTML finalitzat al navegador de l'usuari.

Més endavant s'afegirà l'arquitectura **Service Layer Pattern** o **Fat Service, Skinny Views** afegint les capes de serveis i repositoris.



3. Configuració Inicial

Instal·lació entorn de treball

Els exemples dels projectes que es creen es farà amb **Django 5.0** perquè és la versió (fins ara) més estable per treballar amb **psycopg2** (o **psycopg2-binary**) i **python 3.10**.

Caldrà (amb virtualenv) crear el nostre entorn de treball (ja explicat en classes anteriors). En aquesta part s'explicarà una nova eina més orientada a la programació (**conda**) i es mostraran / explicaran les comandes a utilitzar (l'alternativa està als PowerPoints - **python3 -m**):

Instal·lar django 5.0

```
conda install django=5.0
```

```
conda install -c conda-forge django=5.0
```

en aquest cas **-c conda-forge** es demana que instal·li django 5.0 de la botiga **conda-forge** perquè en la botiga estàndard no la troba.

Instal·lar psycopg2-binary

```
conda install psycopg2-binary
```

Llistar les llibreries instal·lades a l'entorn

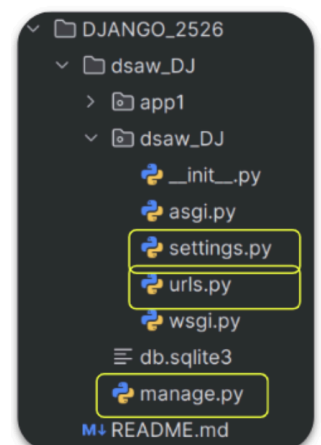
```
conda list
```

Amb aquesta comanda es pot veure totes les llibreries instal·lades amb conda. Si es vol instal·lar alguna altra llibreria només cal posar **conda install** o **conda install -c conda-forge**.

Creació del Projecte

Per iniciar un projecte s'utilitza la comanda: **django-admin startproject <nom_projecte>**. Amb aquesta comanda es crea una carpeta amb els següents arxius:

- **manage.py**: Línia d'ordres per a tasques administratives.
 - **python3 manage.py runserver** ⇒ executa el projecte al port standard.
 - **python3 manage.py runserver 8080** ⇒ executa el projecte al port 8080.
 - **python3 manage.py startapp <nomapp>** ⇒ crear una aplicació nova



- **settings.py:** Configuració global del projecte.
 - Afegir cada aplicació creada a la secció **INSTALLED_APPS**.
 - Afegir la configuració de la base de dades a la secció **DATABASES**.
 - Afegir templates a la secció **TEMPLATES**.
- **urls.py:** Llista de rutes i vistes del projecte.
 - Ubicat a la carpeta del projecte ⇒ per trucar a **urls.py** de les aplicacions.

4. Gestió d'Aplicacions

Per crear una aplicació dins del projecte s'usa la comanda **python3 manage.py startapp <nom_app>**. Aquesta comanda crea una altra carpeta amb el nom indicat a **<nom_app>** i dintre conté 5 arxius (**admin.py**, **apps.py**, **models.py**, **tests.py** i **views.py**). La funcionalitat de cada arxiu s'anirà descobrint al llarg d'aquesta documentació.

Nota important: Cal registrar la nova app a l'arxiu **settings.py** del **projecte** dins de la llista

```

31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'app1',
41 ]
  
```

```

1 from django.apps import AppConfig
2
3
4 class App1Config(AppConfig):
5     default_auto_field = 'django.db.models.BigAutoField'
6     name = 'app1'
7
  
```

default_auto_field = 'django.db.models.BigAutoField' ⇒ S'està configurant el tipus d'id de forma genèrica dintre de l'aplicació. Això permet no haver de crear a cada model l'id corresponent (**id=models.BigAutoField(primary_key=True)**). Django ho farà automàticament.

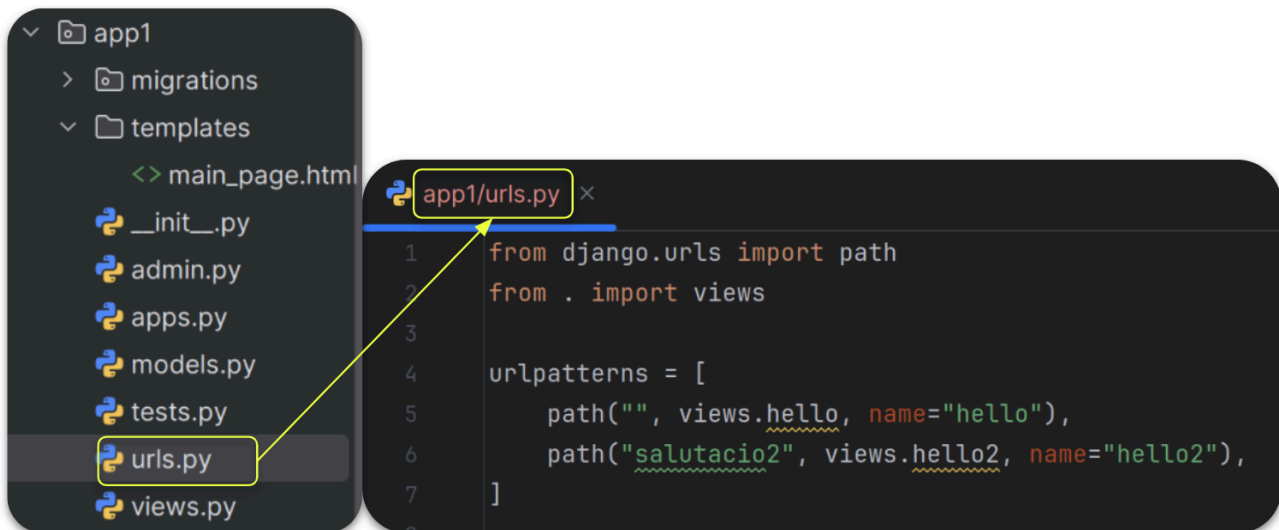
Aquest registre a **settings.py** fa que Django pugui reconèixer **app1**, com una aplicació nova i pot fer el seguiment de tots els seus arxius. Sense aquest registre, Django mai trobarà i reconeixerà aquesta aplicació creada.

5. Routes / urls.py

En l'arxiu **urls.py** del projecte caldrà afegir l'aplicació nova creada com s'indica a la [documentació](#). Caldrà afegir el mètode `include`, ja que es truca a l'arxiu `urls.py` de l'aplicació.



En cada **aplicació** creada caldrà crear l'arxiu **urls.py** en el que s'hi indica la trucada al mètode segons la ruta que vingui de la consulta. Per crear aquest arxiu només cal clicar amb botó dret la carpeta de l'aplicació (**app1**) i crear un nou arxiu de Python de nom **urls.py**. En aquest arxiu s'hi redirigeixen les consultes segons el path indicat (endpoint). Aquestes consultes truquen el mètode de les **views** corresponent. Extret de ➔.



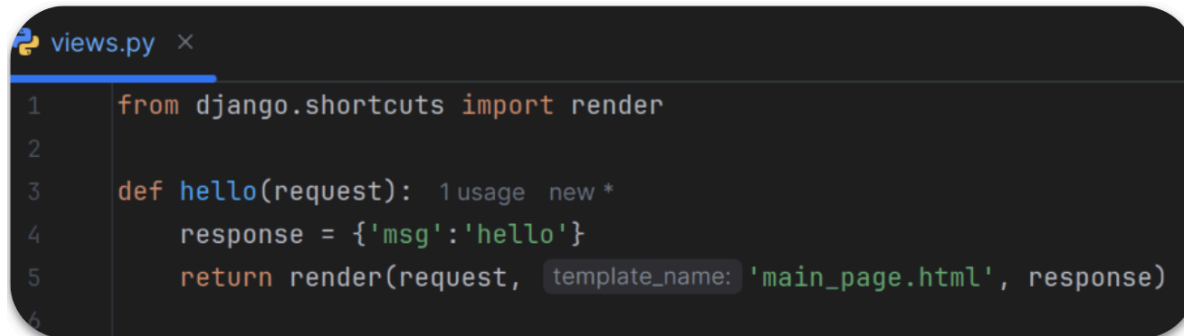
Què està passant?

Si l'usuari va al path **localhost:8000/salutacio2**, Django trucarà al mètode **hello2** de l'arxiu **views**. **urlpatterns** és un llistat de paths amb les seves trucades respectives als mètodes de **views**.

6. Views

Al fitxer **views.py** ubicat a la carpeta de l'aplicació s'hi creen els mètodes i la seva lògica per donar resposta a la consulta.

Cal indicar la informació i el template (plantilla en html) on s'enviarà la informació a renderitzar.

A screenshot of a code editor showing the contents of a file named 'views.py'. The code is written in Python and includes a comment 'usage new *'. It imports the 'render' function from 'django.shortcuts' and defines a 'hello' function that takes a 'request' object as an argument. Inside the 'hello' function, a dictionary 'response' is created with a key 'msg' and a value 'hello'. The function then returns the result of 'render(request, template_name: 'main_page.html', response)'.

```
1 from django.shortcuts import render
2
3 def hello(request): 1 usage new *
4     response = {'msg': 'hello'}
5     return render(request, template_name: 'main_page.html', response)
6
```

S'utilitzarà el mètode render de django en el que s'hi indica la plantilla a on s'enviarà la informació treballada al mètode. En aquest exemple, es renderitza la informació de **response** a la plantilla (template) **main_page.html**.

7. Plantilles (Templates) i Arxius Estàtics

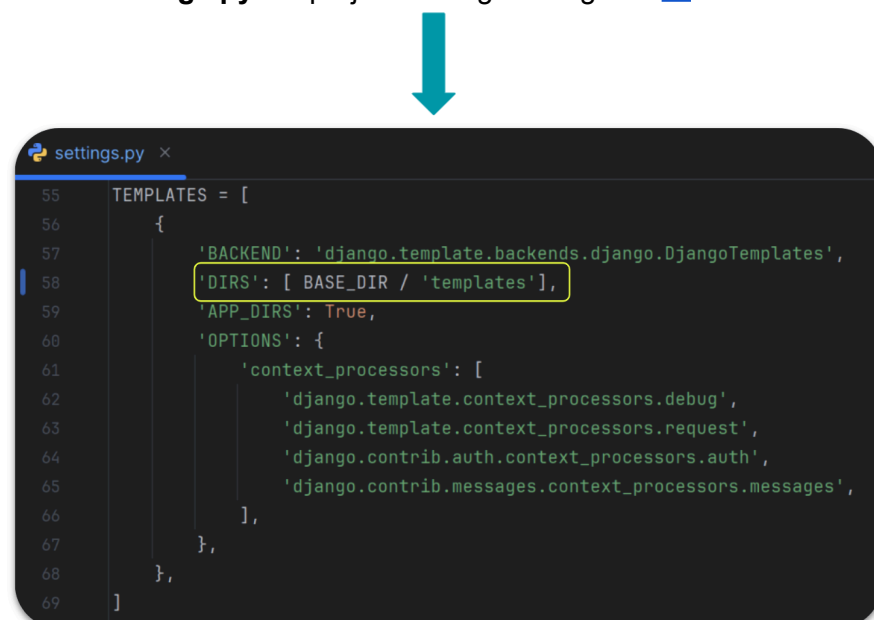
Per mostrar la informació extreta de la base de dades cal crear les plantilles d'HTML. Per a poder treballar amb html, caldrà crear la carpeta **templates** en diversos llocs del projecte de Django. Es

creen a:

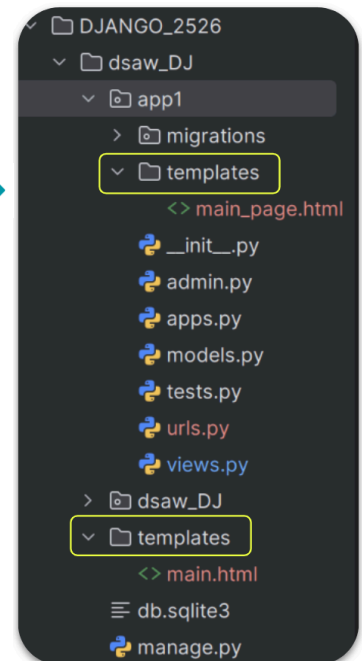
- El directori principal del projecte.
- En cada aplicació.

Es crea la carpeta com s'ha creat **urls.py**, però seleccionant **new > Directory**.

Perquè Django detecti aquestes noves carpetes creades de nom **templates**, caldrà anar a **settings.py** del projecte i afegir el següent ➡:

A screenshot of a code editor showing the 'TEMPLATES' configuration in 'settings.py'. The configuration is a list containing a dictionary. The dictionary has keys for 'BACKEND', 'DIRS', 'APP_DIRS', and 'OPTIONS'. The 'DIRS' value is a list containing 'BASE_DIR / 'templates'', which is highlighted with a yellow box. The 'OPTIONS' dictionary includes a 'context_processors' list with several Django default processors.

```
55 TEMPLATES = [
56     {
57         'BACKEND': 'django.template.backends.django.DjangoTemplates',
58         'DIRS': [ BASE_DIR / 'templates' ],
59         'APP_DIRS': True,
60         'OPTIONS': {
61             'context_processors': [
62                 'django.template.context_processors.debug',
63                 'django.template.context_processors.request',
64                 'django.contrib.auth.context_processors.auth',
65                 'django.contrib.messages.context_processors.messages',
66             ],
67         },
68     },
69 ]
```



8. Lògica Python amb Django (Tags) ➡

Per poder mostrar les dades que s'extreuen de la base de dades en una plantilla HTML si treballem amb Python, Django té una eina anomenada **TAGS** que ens permeten poder treballar amb aquestes dades i mostrar-les a l'HTML.

Des del mètode corresponent de **views** cal indicar amb **render** la plantilla (html) i la informació a renderitzar. Hi ha 2 tipus de tags:

- **{% xxx %}** » quan calgui treballar a l'html amb condicionals, bucles i blocs.
- **{{ xxx }}** » quan calgui accedir a les dades dintre de les variables passades a través del **render**.

En la plantilla (**templates > main.html**) del projecte caldrà afegir el següent codi:

```
<> main.html x
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title> TEMPLATE PRINCIPAL DEL PROJECTE </title>
6  </head>
7  <body>
8      <h1>Plantilla Projecte</h1>
9      {% block content %}
10
11      {% endblock %}
12  </body>
13  </html>
```

Amb els tags **{% block content %}** i **{% endblock %}** s'indica al projecte que aquest document serà la plantilla general en el que es renderitzaran altres templates (d'aplicacions) dintre d'aquest bloc.

Això vol dir, que a la resta de templates que apuntin a aquesta plantilla no se li ha de posar totes les etiquetes d'un HTML com es veu en el següent template de l'aplicació:

```
<> main_page.html x
1  {% extends 'main.html'%}
2
3  {% block content %}
4  <h1>Es mostra llistat d'usuaris des de main_page</h1>
5      <ul>
6          {% for k,v in msg.items %}
7              <h3>{{ k }}</h3>
8              <li>id: {{ v.id }}</li>
9              <li>name: {{ v.name }}</li>
10             <li>email: {{ v.email }}</li>
11             <li>rol: {{ v.rol }}</li>
12             </br>
13             {% endfor %}
14         </ul>
15     {% endblock %}
```

El tag **{% extends 'main.html' %}** indica que aquest template, i tota la informació que hi contingui, es renderitzarà a la plantilla **main.html** (del projecte).

Es renderitzarà tota la informació que estigui dintre del bloc **{% block content %}** i **{% endblock %}**.

Fixar-se com s'utilitzen els diferents tags pel bucle i per l'accés a les dades.



La informació de cada usuari es treballa dintre del **views** de l'aplicació. En aquest exemple, per fer-ho ràpid s'extreu la informació d'una llista o diccionari, però s'acabarà extraient la informació d'una base de dades.

user_list és un diccionari que es guarda dintre de **info** i s'envia per paràmetre a **render** conjuntament amb la plantilla a on s'ha d'enviar aquesta informació.

```
views.py x
7
8 def users(request): 1 usage  👤 roger
9     user_list = {
10         'user1':{
11             'id':1,
12             'name':'Roger',
13             'email':'roger.sobrino@iticbcn.cat',
14             'rol':'teacher',
15         },
16         'user2': {
17             'id': 2,
18             'name': 'Josep Oriol',
19             'email': 'joriol.roca@iticbcn.cat',
20             'rol': 'teacher',
21         },
22         'user3': {
23             'id': 3,
24             'name': 'Pedro',
25             'email': 'Pedro@pedro.cat',
26             'rol': 'student',
27         },
28     }
29     info = {'msg':user_list}
30     return render(request, template_name: 'main_page.html', info)

urls.py x
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     #path("", views.hello, name="hello"),
6     path("salutacio2", views.hello2, name="hello2"),
7     path("users/", views.users, name='users'),
8     path("users_list/", views.usersList, name='u_list'),
9 ]
```

Ara només cal executar el projecte amb la comanda **python3 manage.py runserver** o **py3 manage.py runserver** per veure l'execució del projecte com a la imatge anterior.

Amb aquesta informació ja es pot començar amb la primera activitat de nom **[Aav5_BE] - RA4_RA5_Django mvt + tags**.

Imatge general dels fitxers implicats en l'execució del projecte per l'exemple:

```
<> main_page.html x
1 {% extends 'main.html'%}
2
3 {% block content %}
4 <h1>Es mostra llistat d'usuaris des de main_page</h1>
5     <ul>
6         {% for k,v in msg.items %}
7         <h3>{{ k }}</h3>
8         <li>id: {{ v.id }}</li>
9         <li>name: {{ v.nom }}</li>
10        <li>email: {{ v.email }}</li>
11        <li>rol: {{ v.rol }}</li>
12    </br>
13    {% endfor %}
14    </ul>
15 {% endblock %}

<> main.html x
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title> TEMPLATE PRINCIPAL DEL PROJECTE </title>
6 </head>
7 <body>
8     <h1>Plantilla Projecte</h1>
9     {% block content %}
10
11     {% endblock %}
12 </body>
13 </html>

views.py x
7
8 def users(request): 1 usage 1 roger *
9     user_list = {
10         'user1':{
11             'id':1,
12             'name':'Roger',
13             'email':'roger.sobrino@iticbcn.cat',
14             'rol':'teacher',
15         },
16         'user2': {
17             'id': 2,
18             'name': 'Josep Oriol',
19             'email': 'joriol.roca@iticbcn.cat',
20             'rol': 'teacher',
21         },
22         'user3': {
23             'id': 3,
24             'name': 'Pedro',
25             'email': 'Pedro@pedro.cat',
26             'rol': 'student',
27         },
28     }
29
30     info = {'msg':user_list}
31     return render(request, template_name= 'main_page.html', info)

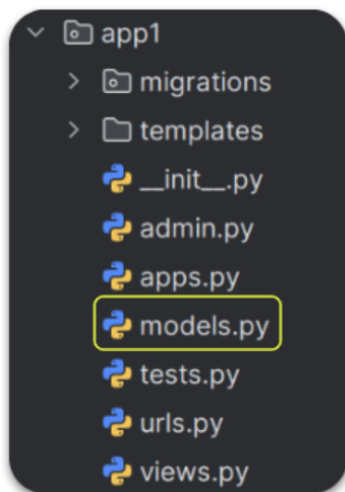
urls.py x
1 from django.urls import path
2 from . import views
3
4 urlpatterns = [
5     #path("", views.hello, name="hello"),
6     path("salutacio2", views.hello2, name="hello2"),
7     path("users/", views.users, name='users'),
8 ]
```


RESUM

1. Totes les aplicacions que es crein caldrà afegir-les a l'arxiu de configuració **settings.py** del projecte.
2. A l'arxiu **urls.py** del projecte caldrà afegir la url que apunti a l'aplicació.
3. En l'arxiu **urls.py** de l'aplicació s'hi torben els paths (urls) que apunten als mètodes de l'arxiu **views.py**.
4. En l'arxiu **views.py** s'hi creen els mètodes que s'encarreguen de **renderitzar** la informació a la plantilla (**template**) indicada.
5. Cada app té els seus **templates** que renderitzen a la plantilla del projecte.
6. Per renderitzar informació en les plantilles s'usen els **tags** de Django (`{% %}`, `{{ }}`)

9. Models ➡

A l'arxiu **models.py** de l'aplicació s'hi creen les taules i els seus atributs per a que es generin a la base de dades (postgresql).



Seguint l'exemple indicat a la [documentació](#) de Django es crea una taula person a l'arxiu **models.py**.

Aquest model serà el que, a través de l'ORM propi de Django (**Django ORM**) crei les taules a la base de dades.

En Aquest model se li indicarà un nom i un tipus de dada. Per consultar el tipus de dada a assignar a l'atribut del model visitar l'[enllaç](#).

Exemple de model:

```
from django.db import models

class Person(models.Model):
    f_name = models.CharField(max_length=30)
    l_name = models.CharField(max_length=30)
    age = models.IntegerField()
    height = models.DecimalField(max_digits=3, decimal_places=2)
```

Per poder comunicar-se amb una base de dades caldrà crear un servei de base de dades amb **docker**. L'arxiu **docker-compose.yml** està ubicat al powerpoint enllaçat d'aquest apartat.

A part caldrà anar a l'arxiu **settings.py** i modificar la secció **DATABASES** i afegir el que es recomana a la [documentació](#) oficial amb les dades de connexió del docker.

```
DATABASES = {  
    "default": {  
        "ENGINE": "django.db.backends.postgresql",  
        "NAME": "",  
        "USER": "",  
        "PASSWORD": "",  
        "HOST": "127.0.0.1",  
        "PORT": "5432",  
    }  
}
```

Els apartats **name**, **user** i **password** s'hi ha de posar les dades de connexió indicades al vostre docker.

Un cop es té afegit la connexió a **DATABASES**, vinculat al docker i creada la nostra taula a **models.py**, només caldrà verificar el seu funcionament seguint aquestes passes:

- Crear les migracions

```
python3 manage.py makemigrations
```

```
Migrations for 'app1':  
app1/migrations/0001_initial.py  
- Create model Person
```

- Migrar la taula

```
python3 manage.py migrate
```

```
Operations to perform:  
Apply all migrations: admin, app1, auth, contenttypes, sessions  
Running migrations:  
Applying contenttypes.0001_initial... OK  
Applying auth.0001_initial... OK  
Applying admin.0001_initial... OK  
Applying admin.0002_logentry_remove_auto_add... OK  
Applying admin.0003_logentry_add_action_flag_choices... OK  
Applying app1.0001_initial... OK  
Applying contenttypes.0002_remove_content_type_name... OK  
Applying auth.0002_alter_permission_name_max_length... OK  
Applying auth.0003_alter_user_email_max_length... OK  
Applying auth.0004_alter_user_username_opts... OK  
Applying auth.0005_alter_user_last_login_null... OK  
Applying auth.0006_require_contenttypes_0002... OK  
Applying auth.0007_alter_validators_add_error_messages... OK  
Applying auth.0008_alter_user_username_max_length... OK  
Applying auth.0009_alter_user_last_name_max_length... OK  
Applying auth.0010_alter_group_name_max_length... OK  
Applying auth.0011_update_proxy_permissions... OK  
Applying auth.0012_alter_user_first_name_max_length... OK  
Applying sessions.0001_initial... OK
```

La primera vegada que es fa la migració es veuen moltes taules que s'estan migran, però només una és la que heu creat **app1.001.initial...** En la següent imatge es veu com s'han creat les taules a postgresQL

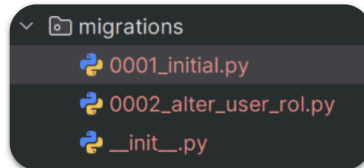
```
Tables (11)  
> app1_person  
> auth_group  
> auth_group_permissions  
> auth_permission  
> auth_user  
> auth_user_groups  
> auth_user_user_permissions  
> django_admin_log  
> django_content_type  
> django_migrations  
> django_session
```

- Modificació taula creada

Si la taula ja existeix a la base de dades i vols aplicar-hi qualsevol modificació (com afegir un camp, canviar un límit de caràcters o afegir restriccions), el procediment correcte a Django és el següent:

1. **Modificació del Model:** Primer, es realitzen els canvis necessaris a la classe dins del fitxer **models.py**.
2. **Generació de la Migració:** Sense eliminar cap fitxer preexistent (dintre de migrations), executem la comanda: **python3 manage.py makemigrations**.

Això generarà un nou fitxer d'història (per exemple, **0002_alter_xxxx_xxx.py**) que conté exclusivament les instruccions del canvi realitzat.



3. **Aplicació del Canvi:** Finalment, cal executar la comanda: **python3 manage.py migrate**. Aquesta ordre trasllada el canvi a la base de dades. Django és prou intel·ligent per saber que la taula ja existeix i, en lloc d'intentar crear-la de nou, hi aplicarà una sentència **ALTER TABLE** (com s'observa en el codi de la migració) per modificar-la sense perdre les dades existents. a la base de dades, ja que està fent un **alter** (com bé indica a l'arxiu de la imatge).

- Exemple ús Enum

Es mostra un exemple de com utilitzar enum en Django:

```
class User(models.Model):
    class Rol(models.TextChoices):
        TEACHER = "Teacher"
        STUDENT = "Student"

    name = models.CharField(max_length=30)
    surname = models.CharField(max_length=30)
    age = models.IntegerField()
    rol = models.CharField(max_length=30, choices=Rol.choices)
```

Es crea una classe de rol indicant les opcions (Teacher i Student) per escollir del camp **rol**. Després es crea el camp rol que fa referència a la classe **Rol**. Com a paràmetre se li indica la classe i el mètode **choices**.

Queries Database

Cal saber com crear una consulta per extreure les dades de la base de dades. Amb Fastapi s'usa SQLAlchemy, en canvi, amb django s'usa l'**ORM de Django** a través de les [querysets](#). Aquestes queries segueixen una nomenclatura i cal seguir-la per tal de que la consulta sigui funcional i que Django la pugui interpretar..

`query_var = ModelName.objects.all()`

on:

query_var = variable que conté la resposta de la consulta.

ModelName = nom del model

objects = atribut de l'objecte Model

all() = mètode de la consulta (**all()**, **get()**, **filter()**...)

Per seguir amb l'exemple de **Person** (tenim el model i la base de dades amb la taula **Person**) només cal crear la url de l'aplicació que truqui al mètode de views que s'encarrega de fer la consulta a la taula **Person** de la base de dades i renderitzar aquestes dades a la plantilla (**template**).

```
def getAllUsers(request): 1 usage  👤 roger *
    allUsers = Person.objects.all() ← queryset de Person
    response = {"persons":allUsers}
    return render(request, template_name: 'main_page.html', response)
```

Amb la queryset **Person.objects.all()** s'està enviant una consulta a través de l'**ORM** a la base de dades per obtenir tots els registres existents a la taula **Person**.

Caldrà modificar el template **main_page.html** per tal de poder mostrar la informació continguda en **allUsers**.

```
<ul>
    {% for person in persons %}

    <li>id: {{ person.id }}</li>
    <li>name: {{ person.f_name }}</li>
    <li>email: {{ person.l_name }}</li>
    </br>
    {% endfor %}
</ul>
```

Es mostra llistat d

- id: 1
- name: Roger
- email: Sobrino
- id: 2
- name: Josep Oriol
- email: Roca
- id: 3
- name: Gemma
- email: Sanchez

10. Paràmetres i enllaç ➡.

En aquest apartat es vol mostrar com fer una consulta més específica passant informació per paràmetre a través de la URL i com s'usen els enllaços (**name**) als path per tal de viatjar entre els templates de forma més dinàmica i no usar la URL.

Paràmetres ➡

La intenció ara és mostrar només una registre segons el seu id, per tant, caldrà passar per paràmetre a través de la URL aquest id. Caldrà modificar el path i el mètode per tal de renderitzar un registre al template.

```
urlpatterns = [  
    path("getAllUsers/", views.getAllUsers, name='get_all_users'),  
    path("user/<int:pk>", views.getUserById, name='get_one_user'),  
]
```

```
def getUserById(request, pk): 1 usage new *  
    getUserById = Person.objects.get(id=pk)  
    response = {"person": getUserById}  
    return render(request, template_name='main_page2.html', response)
```

```
<h1>Es mostra llistat d'un Person segon id des de mai  
    <p><b>Id: </b> {{ person.id}}</p>  
    <p><b>Name: </b> {{ person.f_name}}</p>  
    <p><b>Lastname: </b> {{ person.l_name}}</p>  
    <p><b>height: </b> {{ person.height}} m</p>  
{% endblock %}
```

render

Es mostra llistat d

Id: 2

Name: Josep Oriol

Lastname: Roca

height: 2.09 m

Enllaç

Només ens queda crear enllaços per renderitzar templates de forma dinàmica i sense l'ús d'anar modificant les urls de l'explorador.

A cada path caldrà afegir-li un tercer paràmetre de nom **name** indicant-li un nom, el qual es farà referència als enllaços dels templates.

```
urlpatterns = [
    path("getAllUsers/", views.getAllUsers, name='get_all_users'),
    path("user/<int:pk>/", views.getUserById, name='get_one_user'),
]
```

En aquesta imatge s'han definit dos enllaços, un per cada path (**get_all_users** i **get_one_user**)

Es modificarà el template del projecte i de l'aplicació per usar aquests enllaços per mostrar la informació de cada path definit a **urls.py**.

Plantilla Projecte

[Tots els persons](#)

get_all_users

Es mostra llistat de Person des de main_page

- id: 1
- name: Roger
- [mes info](#)
- id: 2
- name: Josep Oriol
- [mes info](#)
- id: 3
- name: Gemma
- [mes info](#)

get_one_user

Plantilla Projecte

[Tots els persons](#)

Es mostra llistat d

Id: 1

Name: Roger

Lastname: Sobrino

height: 1.67 m

get_all_users

[Tornar](#)

A **main.html** (la plantilla del projecte) s'hi afegeix l'enllaç per mostrar tots els registres de la taula **Person**

```
<body>
  <h1>Plantilla Projecte</h1>
  <a href="{% url 'get_all_users' %}"> Tots els persons</a>
  {% block content %}

  {% endblock %}
</body>
```

A **main_page.html** (plantilla de l'aplicació) on s'hi mostren poques dades de cada registre s'hi afegeix l'enllaç per buscar més info per a cada registre:

```
<ul>
    {% for person in persons %}

    <li>id: {{ person.id }}</li>
    <li>name: {{ person.f_name }}</li>
    <li><a href="{% url 'get_one_user' person.id %}">mes info</a></li>
    </br>
    {% endfor %}
</ul>
```

A **main_page2.html** (segona plantilla de l'aplicació) s'hi mostren totes les dades per registre i s'hi afegeix un enllaç per tornar a mostrar tots els registres:

```
<h1>Es mostra llistat d'un Person segon id des de main_page2</h1>
<p><b>Id: </b> {{ person.id}}</p>
<p><b>Name: </b> {{ person.f_name}}</p>
<p><b>Lastname: </b> {{ person.l_name}}</p>
<p><b>height: </b> {{ person.height}} m</p>
<p><a href="{% url 'get_all_users' %}">Tornar</a></p>
```

11. Form ➡

Per crear un formulari per inserir un no usuari (en el nostre exemple un nou registre de **Person**), normalment, es crearia amb html aquest formulari utilitzant les etiquetes corresponents. Amb Django es pot utilitzar el mateix model **Person** per crear el formulari segons els atributs i els tipus de dades creats.

Per crear aquest formulari de **Person** (l'ordre és indiferent):

- Es crea el template per renderitzar el formulari d'inscripció de **Person** segons el model creat a **models.py**.
- Es crea el mètode (dintre de **views.py**) per renderitzar la informació a la plantilla.
- Es crea el path a **urls.py**.
- Es crea un nou arxiu de nom **forms.py** dintre de l'aplicació i s'hi especifica quins camps (de **Person**) es volen mostrar al formulari.

Caldrà crear un arxiu nou de nom **forms.py** dintre de l'aplicació i afegir-hi el següent codi:

```
from django.forms import ModelForm
from .models import Person

class PersonForm(ModelForm):
    class Meta:
        model = Person
        fields = '__all__'
        #fields = ['f_name', 'l_name']
```

S'haurà d'importar **ModelForm** i crear la classe amb nom inventat. La part important és la part de **field** on s'indicarà amb **all** o amb una llista els camps a utilitzar del model en el formulari.

Amb **class Meta** no cal crear, a la classe del ModelForm, el model altra vegada. Amb **model = Person** Django agafa les dades i el seu tipus (del model creat) per crear el formulari segons els camps indicats a **fields**.

En **views.py** es crea el mètode per utilitzar aquesta classe **PersonForm** i renderitzar el formulari a la plantilla especificada a **render**.

```
from django.urls import path
from . import views

urlpatterns = [
    path("getAllUsers/", views.getAllUsers, name='get_all_users'),
    path("user/<int:pk>/", views.getUserById, name='get_one_user'),
    path("formulari/", views.person_form, name='formulari'),
]
```

```
from django.forms import ModelForm
from .models import Person

class PersonForm(ModelForm):
    class Meta:
        model = Person
        fields = '__all__'
        #fields = ['f_name', 'l_name']
```

```
def person_form(request):
    form = PersonForm()
    context = {"formulari": form}
    return render(request, template_name='formulari.html', context)
```

```
{% extends 'main.html' %}

{% block content %}
<h1>FORMULARI DE PERSON</h1>

<form action="" method="POST">
    {% csrf_token %}
    {{ formulari }}
    <input type="submit">
</form>
{% endblock %}
```


12. Crud ➡

A continuació es mostra com es faria un crud amb Django. El **Read** ja s'ha mostrat en anteriors punts, per tant es mostrarà com fer el **Create**, **Update** i **Delete**. Per veure aquesta informació a la documentació de Django cal una búsqueda més exhaustiva. Tanmateix es comparteix un [enllaç](#) on surten alguns mètodes que farem servir.

Create

Per crear un nou registre de **Person** (per seguir amb l'exemple anterior) cal crear un html on poder afegir-hi les dades d'aquest objecte i guardar-la a la base de dades. S'usa el concepte vist a **form** en l'anterior punt.

```
def person_form(request): 1 usage 1 roger *
    form = PersonForm()

    if request.method == "POST":
        form = PersonForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('main_page.html')

    context = {"formulari":form}
    return render(request, template_name: 'formulari.html', context)
```

Amb **.is_valid()** Django revisa si les dades són vàlides segons el tipus de model i les dades que s'envien a través de formulari.

El mètode **save()** permet guardar les dades del formulari a la taula de la base de dades corresponent al model emprat per crear el formulari.

Cal importar **redirect** a `from django.shortcuts import render, redirect` per redirigir el projecte a la plantilla que es vulgui renderitzar després d'aquesta acció.

Update

Per fer la modificació a un registre caldrà crear un mètode per buscar aquell registre per id, modificar la dada o les dades i usar el mètode **save()** per guardar la informació del registre. Cal tenir en compte que sempre que es crei un mètode nou a **views.py** caldrà tenir en compte que per trucar-lo s'haurà d'afegir una url (si s'utilitza la url per trucar aquest mètode).

```
def update_person(request, pk): 1 usage 1 new *
    person_id = Person.objects.get(id=pk)
    form = PersonForm(instance=person_id)

    if request.method == "POST":
        form = PersonForm(request.POST, instance=person_id)
        if form.is_valid():
            form.save()
            return redirect('get_all_users')

    context = {"formulari":form}
    return render(request, template_name: 'formulari.html', context)
```

A **person_id** s'hi guarda l'objecte de Person amb l'id passat per paràmetre a la url.

form = PersonForm(instance=person_id) es carrega la informació de **person_id** a dintre dels atributs el formulari per mostrar-ho al template **formulari.html**.

Dins de la funció, el bloc `if request.method == "POST"`: gestiona el moment en què l'usuari envia el formulari amb els nous canvis. En aquesta línia, `form = PersonForm(request.POST, instance=person_id)` és important, ja que recull les dades introduïdes (`request.POST`) i les vincula directament a l'objecte existent (`instance=person_id`).

Un cop el mètode `is_valid()` confirma que les dades compleixen les regles del model, l'execució de `form.save()` no crea un registre nou, sinó que actualitza la informació de la persona específica a la base de dades.

Caldrà modificar l'html per afegir el botó d'editar per a trucar al mètode `update_person`

```
{% extends 'main.html'%}

{% block content %}
<h1>Es mostra llistat d'un Person segon id des de main_page2</h1>
    <tr>
        <td><b>Id: </b> {{ person.id}}</td>
        <td><b>Name: </b> {{ person.f_name}}</td>
        <td><b>Lastname: </b> {{ person.l_name}}</td>
        <td><b>height: </b> {{ person.height}} m</td>
        <td><a href="{% url 'update_p' person.id %}" >EDITAR</a></td>
    </tr>
    <p><a href="{% url 'get_all_users' %}">Tornar</a></p>
{% endblock %}
```

Delete

Per eliminar un registre caldrà crear el seu mètode a `views.py`, amb una estructura semblant a la de l'update.

```
def delete_user(request, pk):
    new *
    person_id = Person.objects.get(id=pk)

    if request.method == "POST":
        person_id.delete()
        return redirect('get_all_users')

    context = {'person':person_id}
    return render(request, template_name: 'delete_person.html', context)
```

Dintre del bloc `if` s'hi espera una confirmació a través d'un formulari per a validar el borrar de la base de dades. Aquesta confirmació ve de la plantilla indicada al `render`.

```
{% extends 'main.html' %}

{% block content %}
<h1>CONFIRMACIÓ ELIMINACIÓ PERSON</h1>

<form action="" method="POST">
    {% csrf_token %}
    <p>Segur que vols eliminar a {{ person.f_name }} ?</p>
    <button><a href="{% url 'get_all_users' %}">TORNAR</a></button>
    <input type="submit" value="CONFIRM">
</form>

{% endblock %}
```

Codi `delete_person.html` per a confirmar que es vol eliminar l'usuari.

```
{% extends 'main.html'%}

{% block content %}
<h1>Es mostra llistat d'un Person segon id des de main_page2</h1>
    <tr>
        <td><b>Id: </b> {{ person.id}}</td>
        <td><b>Name: </b> {{ person.f_name}}</td>
        <td><b>Lastname: </b> {{ person.l_name}}</td>
        <td><b>height: </b> {{ person.height}} m</td>
        <td><a href="{% url 'update_p' person.id %}" >EDITAR</a></td>
        <td><a href="{% url 'delete_p' person.id %}" >BORRAR</a></td>
    </tr>
    <p><a href="{% url 'get_all_users' %}">Tornar</a></p>
{% endblock %}
```

S'afegeix el botó per eliminar el registre

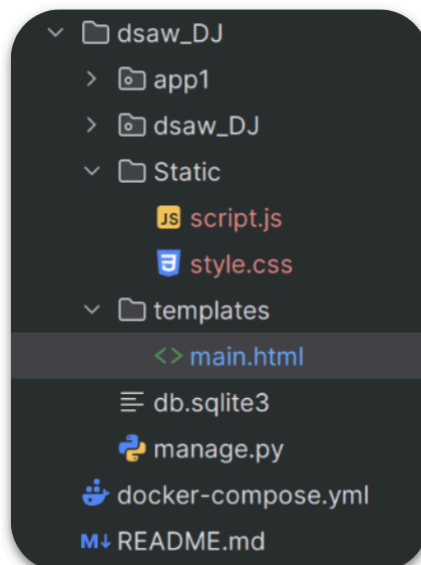
```
from django.urls import path
from . import views

urlpatterns = [
    path("getAllUsers/", views.getAllUsers, name='get_all_users'),
    path("user/<int:pk>/", views.getUserById, name='get_one_user'),
    path("formulari/", views.person_form, name='formulari'),
    path("update_user/<int:pk>", views.update_person, name="update_p"),
    path("deetele_user/<int:pk>", views.delete_person, name="delete_p"),
]
```

S'afegeix la url per passar per paràmetre l'id i per indicar-hi un enllaç

13. Static ➡

Per a poder utilitzar fitxers css i/o js cal crear una carpeta nova de nom **static** al directori principal del projecte. A dins de la carpeta **static** afegir tots els css i js.



A **settings.py** ja està definit l'ús d'una carpeta de nom static, però si no funciona es pot afegir el següent codi:

```
STATIC_URL = 'static/'

#Si no funciona el direccionament anteriori, afegir el següent
STATICFILES_DIRS = [
    BASE_DIR / "static",
]
```

Sempre que es vulgui aplicar un fitxer js i/o css a un html de Django, caldrà utilitzar els tags **{% %}** per buscar aquests fitxers.

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title> TEMPLATE PRINCIPAL DEL PROJECTE </title>
    <link rel="stylesheet" href="{% static 'style.css' %}">
</head>
<body>
    <h1>Plantilla Projecte</h1>
    <a href="{% url 'get_all_users' %}"> Tots els persons</a>
    {% block content %}

    {% endblock %}

    <script src="{% static 'script.js' %}"></script>
</body>
</html>
```

GIT

COMANDES GIT

Comanda per afegir un arxiu d'una branca a una altra branca. Caldrà situar-se a la branca on es vol afegir aquell arxiu i utilitzar la comanda:

Per exemple; tinc 3 branques (main, crud i config). Es vol afegir l'arxiu **settings** de la branca config a la branca crud. Cal situar-se a la branca crud: **git checkout crud** i un cop a la branca crud, caldrà posar la següent comanda **git checkout config -- path/settings.py** on path correspon al camí de carpetes del projecte per arribar a la ubicació de l'arxiu a copiar.